*"@Endpoint and @PayloadRoot turn a signed contract into working code."*

MODULE 24

# SOAP Web Service Endpoints and Request Mapping

Build, secure, and integrate contract-first SOAP web services using Spring Web Services (Spring WS) endpoints.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

# WHY SOAP REMAINS IMPORTANT

*SOAP continues to power mission-critical integrations where security, reliability, and standards matter most.*

While REST has become popular, SOAP continues to be critical in many enterprise environments.

**Enterprise-Grade Security —** built-in support for WS-Security standards provides message-level security, encryption, signatures, and authentication.

**Formal Contract and Strong Typing —** WSDL and XML Schema enforce strict contracts, ensuring clarity, validation, and robust integration.

**Reliability and Standards —** uses proven protocols (HTTP, SMTP, etc.) and standards supported by major vendors and organizations.

**Enterprise and Legacy Integration —** ideal for integrating with legacy systems, mainframes, and heterogeneous platforms.

**Governance and Compliance —** meets regulatory and compliance requirements in industries like banking, healthcare, and insurance.

**SOAP IS STILL RELEVANT WHERE...**

- Secure by Design
- Standards Based
- Reliable Messaging
- Compliance Ready
- Legacy System Integration
- Enterprise Adoption

**KEY TAKEAWAY** SOAP continues to power mission-critical integrations where security, reliability, and standards matter most.

# MODULE LEARNING OBJECTIVES

*By the end of this module, you will be able to design, develop, and secure SOAP web services using Spring WS.*

**Review SOAP Fundamentals —** recall SOAP architecture, messages, WSDL, and XML Schema concepts.

**Understand Spring Web Services —** explain the purpose, architecture, and core components of Spring WS.

**Apply Contract-First Development —** design SOAP services using WSDL and XML Schema before writing code.

**Build SOAP Endpoints —** create and configure endpoints using @Endpoint and @PayloadRoot.

**Process SOAP Requests —** implement the request/response processing workflow and message lifecycle.

**Work with XML —** perform XML marshalling and unmarshalling of SOAP payloads.

**Handle SOAP Faults —** structure and translate business errors into standards-compliant SOAP faults.

**Implement WS-Security —** apply WS-Security concepts, including UsernameToken, to secure SOAP services.

**Integrate Enterprise SOAP Services —** consume and expose SOAP services in enterprise environments.

**Follow Enterprise Best Practices —** apply best practices for reliable, secure, maintainable SOAP services.

**KEY TAKEAWAY** These objectives build the skills to design, build, secure, and integrate production-grade SOAP endpoints with Spring WS.

# SOAP ARCHITECTURE RECAP

*A brief refresher -- Module 13 already covered SOAP envelope, headers, faults, and versioning in depth.*

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in a decentralized, distributed environment.

## SOAP MESSAGE STRUCTURE

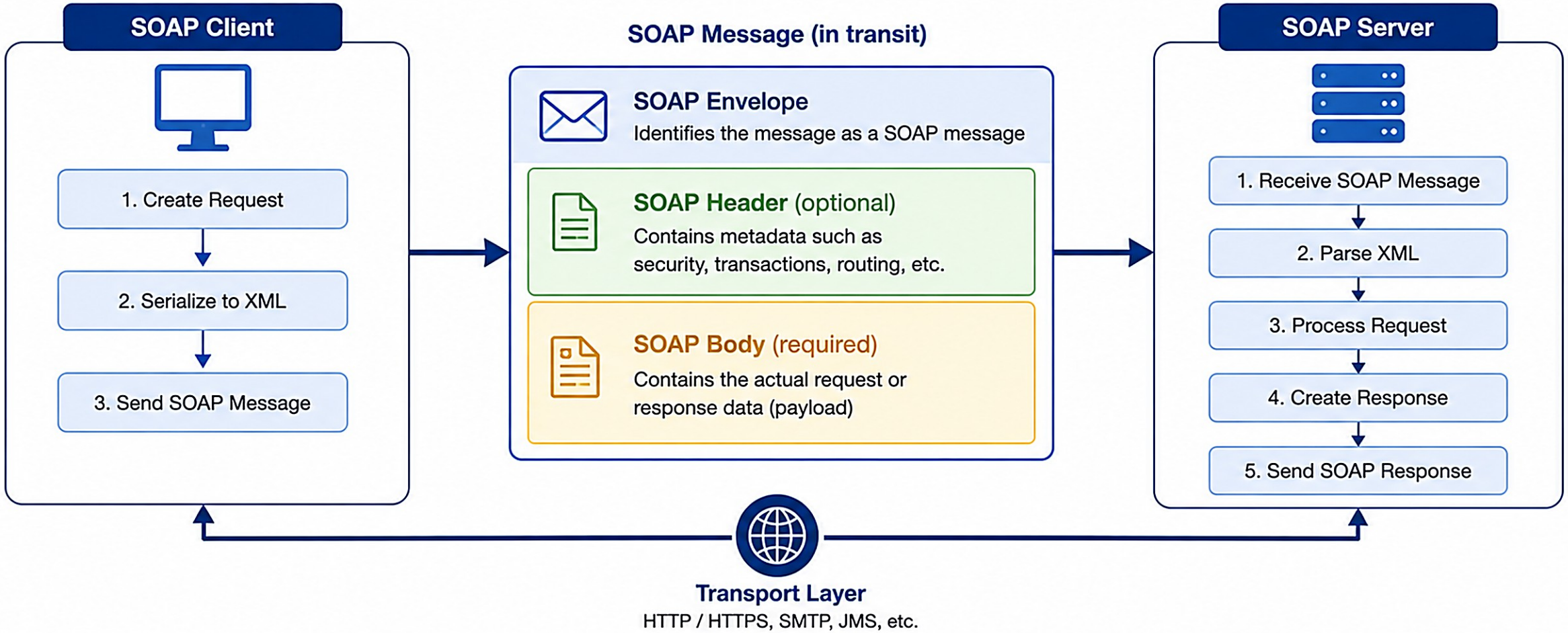
| Part | Role |
|---|---|
| **SOAP Envelope** | The root element that defines the XML document as a SOAP message. |
| **SOAP Header (Optional)** | Carries metadata such as security, transactions, routing, and other context. |
| **SOAP Body (Mandatory)** | Carries the actual request or response data (the payload). |
| **Transport Layer** | Typically HTTP/HTTPS, SMTP, or JMS -- moves the SOAP message between client and server. |

## HOW IT WORKS


**KEY TAKEAWAY** SOAP's formal envelope, header, and body structure keeps it standards-based and enterprise-ready.

# SOAP Architecture

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in the implementation of web services over a network.




**Key Points**

- SOAP uses XML for message format.
- Messages are transferred over standard protocols like HTTP/HTTPS.
- SOAP is platform and language independent.

# WSDL REVIEW (1 OF 2)

*WSDL (Web Services Description Language) is an XML-based language that describes a SOAP service and how to access it.*

## PURPOSE OF WSDL

- Describes what the service does, how it can be accessed, where it is located, and what data formats it uses.

## KEY COMPONENTS OF WSDL

| Component | Defines |
|---|---|
| **types** | Reusable data types used by the service, defined using XML Schema (XSD). |
| **message** | The input and output messages for the operations. |
| **portType** | The interface -- the set of operations (methods) the service exposes. |
| **binding** | The protocol (SOAP) and data format (e.g. document/literal) used to communicate. |
| **service** | The actual endpoint URL where the service can be accessed. |

**KEY TAKEAWAY** types -> message -> portType -> binding -> service: WSDL builds from reusable data types up to a live, addressable endpoint.

# WSDL (Web Services Description Language)

WSDL is an XML based contract that describes a web service. It defines what the service does, how to access it, and what the request and response messages look iike.

## WSDL Structure Overview

**Types**
Defines the data types used in requests and responses using XML Schema (XSD).
Data Definitions

**Messages**
Defines the input and output messages using the types.
Message Formats

**Port Types**
Defines the operations (methods) available and the messages used in those operations.
Service Contract

**Bindings**
Specifies the protocol and data format for the operations defined in the port type.
Protocol & Format

**Service**
Defines the endpoint (URL) where the service can be accessed.
Endpoint Location

## WSDL Example (Simplified)

WSDL

```
<definitions name="HelloService"
   targetNamespace="http://example.com/hello"
   xmlns="http://schemas.xmlsoap.org/wsdl/"
   xmlns:tns="http://example.com/hello"
   xmlns:xsd="http://www.w3.org/2001/XMLSchema">

   <!-- Types: Data definitions -->
   <types>
     <xsd:schema targetNamespace="http://example.com/hello">
       <xsd:element name="sayHelloRequest">
         <xsd:complexType>
           <xsd:sequence>
             <xsd:element name="name" type="xsd:string"/>
           </xsd:sequence>
         </xsd:complexType>
       </xsd:element>
       <xsd:element name="sayHelloResponse">
         <xsd:complexType>
           <xsd:sequence>
             <xsd:element name="greeting" type="xsd:string"/>
           </xsd:sequence>
         </xsd:complexType>
       </xsd:element>
     </xsd:schema>
   </types>

   <!-- Messages -->
   <message name="SayHelloRequest">
     <part name="parameters" element="tns:sayHelloRequest"/>
   </message>
   <message name="SayHelloResponse">
     <part name="parameters" element="tns:sayHelloResponse"/>
   </message>

   <!-- Port Type (Operations) -->
   <portType name="HelloPortType">
     <operation name="sayHello">
       <input message="tns:SayHelloRequest"/>
       <output message="tns:SayHelloResponse"/>
     </operation>
   </portType>

   <!-- Binding (Protocol & Format) -->
   <binding name="HelloBinding" type="tns:HelloPortType">
     <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
     <operation name="sayHello">
       <soap:operation soapAction="http://example.com/hello/sayHello"/>
       <input>
         <soap:body use="literal"/>
       </input>
       <output>
         <soap:body use="literal"/>
       </output>
```

## Key Purposes of WSDL

**Contract First**
Defines a clear contract before impliementation.

**Interoperability**
Enables different platforms and languages to integrate.

**Discoverability**
Allows clients to discover service operations and data formats.

**Standardization**
Uses open standards (XML, SOAP, XSD) for enterprise integration.

**Tool Support**
Used by tools to generate client stubs and server skeletons.

## Key Takeaways

- WSDL is the contract of a SOAP web service.
- It defines data types, messages, operations, protocol, and endpoint.
- Clients use WSDL to understand how to interact with the service.
- Essential for enterprise, cross-platform, and loosely coupled systems.

# WSDL REVIEW (2 OF 2)

*A sample WSDL document, how it is used end to end, and why it matters for contract-first integration.*

## Sample WSDL Snippet

```
<definitions name="CustomerService" targetNamespace="http://northstar.com/crm/customer"
             xmlns:tns="http://northstar.com/crm/customer" xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- types --> <types> ... </types>
  <!-- messages --> <message name="GetCustomerRequest"> ... </message>
  <!-- portType --> <portType name="CustomerServicePort"> ... </portType>
  <!-- binding --> <binding name="CustomerBinding" type="tns:CustomerServicePort">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  </binding>
  <!-- service --> <service name="CustomerService">
    <port name="CustomerPort" binding="tns:CustomerBinding">
      <soap:address location="http://localhost:8080/ws"/>
    </port>
  </service>
</definitions>
```

## HOW WSDL IS USED

- Provider Creates WSDL
- Client Reads WSDL
- Client Generates Stub
- Exchange SOAP Messages

## WHY WSDL MATTERS

- Contract-First Dev
- Loose Coupling
- Interoperability
- Legacy Integration

**KEY TAKEAWAY** DefaultWsdl11Definition generates WSDL dynamically from your XSD, so it never drifts from the contract.

# XML SCHEMA REVIEW (1 OF 2)

*XML Schema (XSD) defines the structure, data types, and constraints for XML documents -- the foundation for strong typing in SOAP.*

## KEY CONCEPTS

| Concept | Description |
|---|---|
| **Elements** | Define the structure and hierarchy of XML data. |
| **Data Types** | Define the type of data (string, int, boolean, date, etc.). |
| **Constraints** | Apply rules like minOccurs, maxOccurs, length, pattern, etc. |
| **Complex Types** | Define elements that contain other elements or attributes. |
| **Attributes** | Provide additional information about elements. |

## XML SCHEMA STRUCTURE

1. **xs:schema** — Root element
2. **xs:element** — Defines elements
3. **xs:complexType** — Complex structures
4. **xs:simpleType** — Custom simple types
5. **xs:attribute** — Defines attributes

**KEY TAKEAWAY** XSD is reused by WSDL to define message formats for SOAP operations -- it is the single source of truth for the contract.

# XML Schema (XSD)

XML Schema Definition (XSD) is an XML based language used to describe the structure, data types, and constraints of XML documents.

## XSD Fundamentals

### What is XSD?

XSD defines the legal structure of XML documents. It acts as a blueprint.

### Why Use XSD?

- Ensures data consistency
- Validates XML documents
- Promotes interoperability
- Enables contract-first design

### Built on XML

XSD is itself written in XML and uses namespaces.

### Strong Typing

Supports rich data types (string, int, date, boolean, etc.) and constraints.

## XSD Structure Overview

**Element** — Defines the structure and content

→ **Complex Type** — Groups elements and defines structure

→ **Simple Type** — Defines restrictions on values

→ **Attribute** — Adds metadata to elements

→ **Restriction** — Constrains values of simple types

## XSD Example

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
      targetNamespace="http://example.com/customer"
      xmlns="http://example.com/customer"
      elementFormDefault="qualified">

<!-- Root Element -->
<xs:element name="Customer">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="id" type="xs:int"/>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="email" type="xs:string" minOccurs="0"/>
      <xs:element name="address" type="AddressType"/>
    </xs:sequence>
    <xs:attribute name="status" type="xs:string" use="optional" default="ACTIVE"/>
  </xs:complexType>
</xs:element>

<!-- Complex Type Definition -->
<xs:complexType name="AddressType">
   <xs:sequence>
      <xs:element name="street" type="xs:string"/>
      <xs:element name="city"   type="xs:string"/>
      <xs:element name="country" type="xs:string"/>
   </xs:sequence>
 </xs:complexType>
</xs:schema>
```

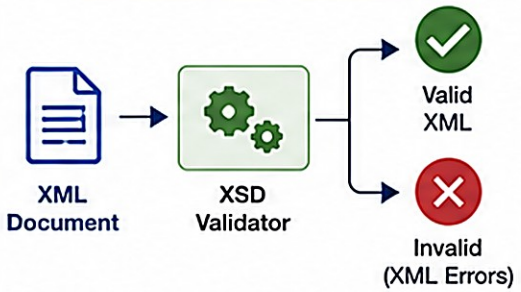
**Legend**

| | |
|---|---|
| Blue | : Elements |
| Green | : Complex Types |
| Orange | : Attributes |
| Purple | : Data Types |
| Gray | : Comments |

## Data Types in XSD

| Category | Examples |
|---|---|
| String Types | xs:string, xs:token, xs:normalizedString |
| Numeric Types | xs:int, xs:long, xs:decimal, xs:float, xs:double |
| Date / Time Types | xs:date, xs:time, xs:dateTime, xs:duration |
| Boolean Type | xs:boolean |

## XSD Validation Flow




## Key Takeaways

- XSD defines the structure and rules for XML data.
- It enables validation, consistency, and reliability.
- Essential for contract-first and enterprise integration.
- XSD + WSDL = Strong contract for robust SOAP services.

# XML SCHEMA REVIEW (2 OF 2)

*A sample XSD, its benefits, and where XML Schema is used across SOAP web services.*

**Sample XML Schema (XSD) -- customer.xsd**

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:tns="http://northstar.com/crm/customer"
           targetNamespace="http://northstar.com/crm/customer" elementFormDefault="qualified">
  <xs:simpleType name="CustomerStatus">
    <xs:restriction base="xs:string">
      <xs:enumeration value="PROSPECT"/> <xs:enumeration value="ACTIVE"/>
      <xs:enumeration value="SUSPENDED"/> <xs:enumeration value="CLOSED"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="CustomerType">
    <xs:sequence>
      <xs:element name="customerId" type="xs:string"/>
      <xs:element name="status" type="tns:CustomerStatus"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

## BENEFITS OF XML SCHEMA

- Validates XML
- Enforces Data Types
- Improves Interoperability
- Reduces Integration Errors

## COMMON USE CASES

- Request/Response Structures
- Data Validation
- JAXB Code Generation

**KEY TAKEAWAY** jaxb2-maven-plugin generates Java classes directly from this XSD -- the schema drives the code.

# WHAT IS SPRING WEB SERVICES?

*Spring WS helps you build robust, standards-compliant SOAP web services that interoperate with any platform or language.*

Spring Web Services (Spring WS) is a lightweight, extensible framework for building SOAP-based web services and clients in the Java ecosystem. It simplifies contract-first SOAP development using XML, WSDL, and XSD.

## KEY HIGHLIGHTS

### Standards Based

Built on SOAP, WSDL, XML Schema, and other open standards.

### Contract-First

Service contracts are defined using WSDL and XSD before implementation.

### Lightweight & Flexible

Lightweight vs. full JAX-WS; integrates easily with the Spring ecosystem.

### Enterprise Ready

Supports WS-Security, logging, validation, exception handling, monitoring.

## COMMON USE CASES

- Enterprise System Integration
- B2B / Partner Integrations
- Legacy System Modernization
- Compliance-Driven Solutions
- Cross-Org Data Exchange

**KEY TAKEAWAY** Spring WS helps you build robust, standards-compliant SOAP web services that interoperate with any platform or language.

# SPRING WS ARCHITECTURE

*Spring Web Services provides a flexible, extensible, contract-first framework for building SOAP-based web services.*

| Component | Responsibility |
|---|---|
| **Message Dispatcher** | Receives incoming SOAP requests and dispatches them to the correct endpoint. |
| **Endpoint Adapter** | Bridges the transport layer and the endpoint by invoking the business method. |
| **@Endpoint** | POJO annotated with @Endpoint containing the business logic for the service. |
| **Marshalling / Unmarshalling** | Uses JAXB (or another marshaller) to convert between Java objects and XML. |
| **Message Factory** | Creates and processes SOAP messages and manages SOAP headers and payload. |
| **Transport Layer** | Responsible for delivering messages over HTTP, SMTP, JMS, etc. |

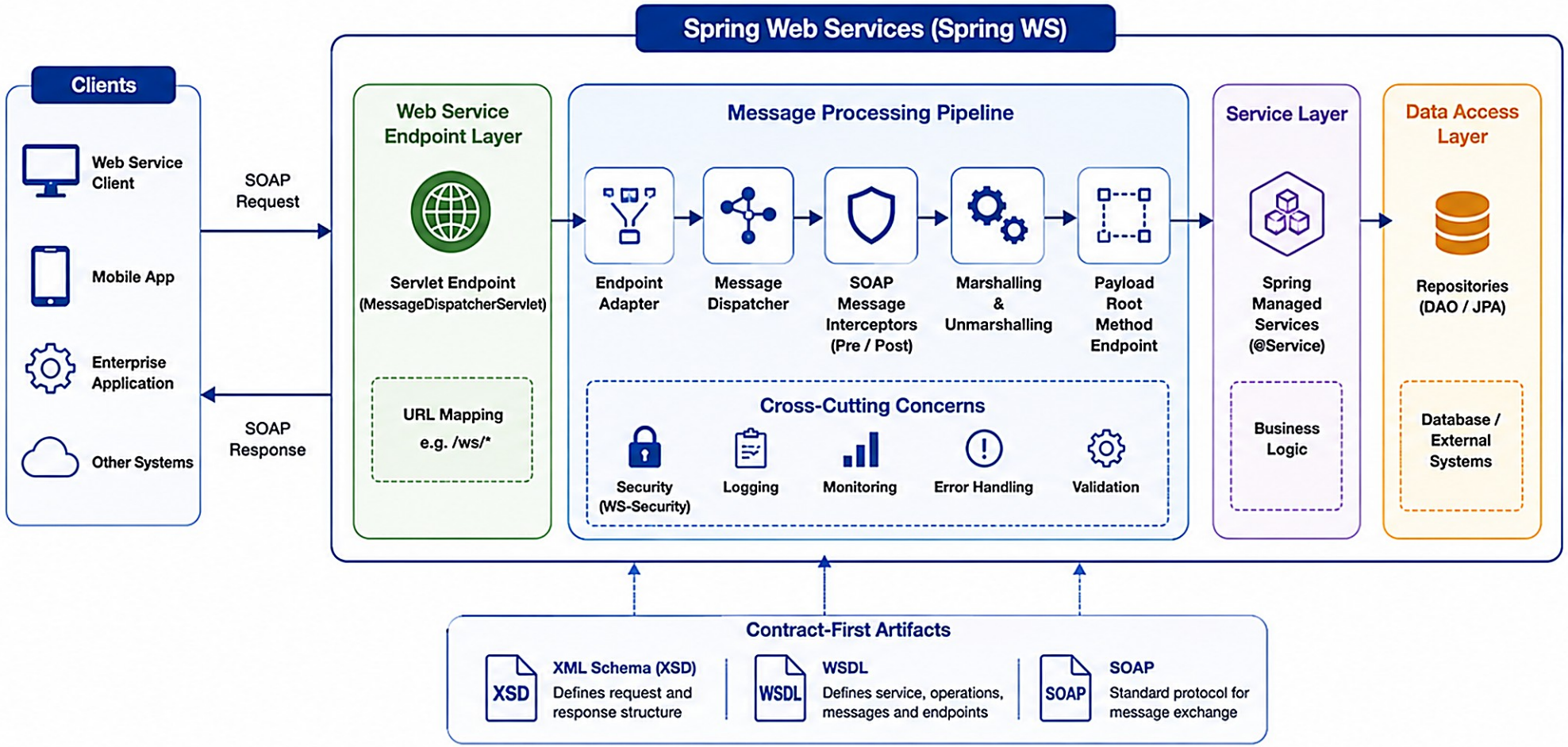
## SOAP REQUEST PROCESSING FLOW

1. **Client Sends**
2. **Transport Receives**
3. **Dispatcher Maps**
4. **Invokes Endpoint**
5. **Endpoint Runs**
6. **Marshals Reply**

**KEY TAKEAWAY** Cross-cutting support (security, validation, logging) wraps every step from Dispatcher through Message Factory.

# Spring WS Architecture

Spring Web Services (Spring WS) provides a lightweight framework for building SOAP-based web services using a contract-first approach.




## Key Characteristics

**Contract-First**
Define XSD and WSDL before implementation.

**Lightweight**
Built on Spring Core, easy to integrate.

**Extensible**
Interceptors, marshallers, validators and more.

**Enterprise Ready**
Supports WS-Security, transactions, logging, monitoring.

**Standards Based**
Uses SOAP, XML, XSD, and WSDL standards.

## Key Takeaways

- Spring WS provides a robust architecture for building SOAP web services.
- It follows a modular pipeline for processing SOAP requests and responses.
- Contract-first approach ensures clear service contracts and strong integration.
- Built-in support for security, monitoring, validation and error handling.

# CONTRACT-FIRST DEVELOPMENT (1 OF 2)

*Contract-First Development means defining the service contract (WSDL and XML Schema) before writing any code.*

The contract acts as an agreement between the service provider and consumers, ensuring a clear and stable interface. After gathering requirements, design the service interface first, agree on the contract, then implement -- this reduces rework and enables parallel development.

## CONTRACT-FIRST DEVELOPMENT PROCESS


The contract (WSDL + XSD) is the single source of truth and remains stable over time -- providers and consumers can then work independently against it.

**KEY TAKEAWAY** In Lab 24 the XSD is authored first (Step 2); the WSDL, JAXB types, and endpoint all follow from it -- never the reverse.

# Contract-First Development

Contract-First Development means designing the service contract (XSD & WSDL) before writing any code.
The contract drives the implementation, ensuring clear agreements and loose coupling.

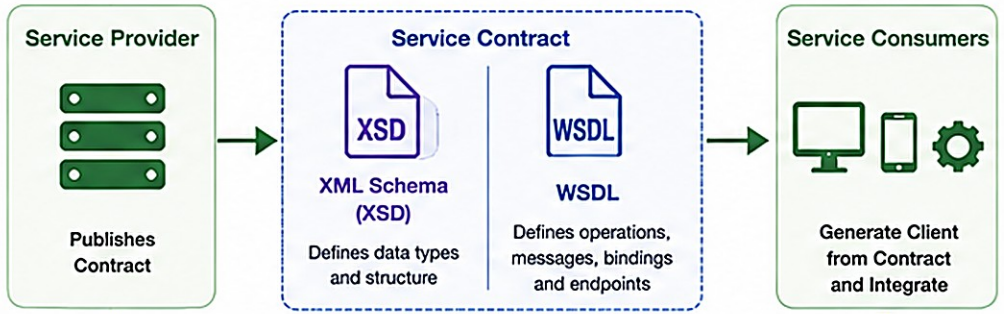
## Contract-First Development Lifecycle

1. **Business Requirements**
   Understand business needs and identify service operations, inputs and outputs.
2. **Design the Contract**
   Design XML Schema (XSD) for data and WSDL for service operations, messages and endpoints.
3. **Review & Validate Contract**
   Validate the XSD and WSDL. Review with stakeholders and ensure the contract meets requirements.
4. **Generate Artifacts & Stubs**
   Generate server skeletons and client stubs from WSDL (e.g., using wsimport, XJC, or IDE tools).
5. **Implement Service**
   Implement the service logic in generated skeletons without changing the contract.
6. **Test with Contract**
   Test requests and responses against the contract to ensure compliance.
7. **Deploy & Integrate**
   Deploy the service and integrate with clients using the published contract.

## Benefits

- **Clear Agreement**
  All parties agree on the contract before implementation.
- **Loose Coupling**
  Changes to implementation do not affect the contract.
- **Early Validation**
  Issues are identified early by validating the contract.
- **Parallel Development**
  Service and client teams can work in parallel.
- **Reusability**
  Contract can be reused across multiple implementrations.

## Contract-First Flow




## Best Practices

- Design for clarity and simplicity.
- Version your contracts.
- Maintain backward compatibility.
- Include security requirements in the contract.
- Review contract changes with all stakeholders.

## Key Takeaways

- Contract-First ensures the contract drives the implementation.
- XSD and WSDL are created, validated and agreed before coding.
- It improves quality, reduces rework and enables better collaboration.
- Changes are managed through versioned contracts, not breaking implementations.

# CONTRACT-FIRST DEVELOPMENT (2 OF 2)

*Key artifacts, benefits, and best practices for contract-first SOAP development.*

## KEY ARTIFACTS

**XSD (XML Schema) —** defines the structure, data types, and constraints of request/response messages.

**WSDL —** describes the service, operations, message formats, and how to access it.

**Service Contract —** the agreement between provider and consumer that should not change frequently.

**Generated Artifacts —** server stubs, client proxies, and data bindings (via wsimport, xjc, or JAXB).

## BENEFITS

- **Clear Agreement**
- **Parallel Development**
- **Reduced Rework**
- **Consistency**
- **Maintainability**

## BEST PRACTICES

- Invest time in designing a robust contract with clear documentation.
- Version your contracts (e.g., v1, v2) and maintain backward compatibility.
- Automate contract validation in the build pipeline.

**KEY TAKEAWAY** A well-designed, versioned contract is the single source of truth for every consumer.

# TRY IT YOURSELF — EXERCISE 1: CONTRACT-FIRST RECALL

*Reinforces: the XSD -> JAXB -> WSDL contract-first order and the Lab 13 link you just covered.*

## STEP / WHAT TO DO

| Step | What To Do |
|---|---|
| **Goal** | Explain why the partner XSD -- not Java classes -- owns the SOAP contract. |
| **File** | notes/contract-first.md (in examples/module-24-exercises/). |
| **Order** | Align your explanation with the XSD -> JAXB -> WSDL order: XSD is the source of truth. |
| **Lab 13 link** | Note that Lab 24 implements Lab 13's customer operations over Spring-WS. |
| **Deliverable** | Contract-first rule and Lab 13 link documented; state you will not author the full XSD yet. |
| **Reference** | exercises/exercise-01-contract-first-recall.md |

## Contract-First Flow (restate this in your own words in notes/contract-first.md)

```
customer.xsd  (hand-authored -- the single source of truth)
   |  mvn generate-sources (jaxb2-maven-plugin)          v  DefaultWsdl11Definition
   v                                                     v
Generated JAXB types (derived, never hand-edited)   Dynamic WSDL /ws/customer.wsdl (never hand-written)
```

**TRY IT NOW** Complete Exercise 1 before continuing -- see exercises/exercise-01-contract-first-recall.md.

# ENDPOINT ARCHITECTURE

*In Spring Web Services, an endpoint is the server-side component that receives, processes, and responds to SOAP requests.*

## WHAT IS AN ENDPOINT?

**Business Function —** an endpoint represents a specific business function exposed as a SOAP web service operation.

**Annotation-Mapped —** handles incoming SOAP requests and maps them to Java methods using annotations.

**Runtime-Managed —** operates within the Spring WS infrastructure for message handling, marshalling, and security.

**Focused & Reusable —** kept focused on request handling and easy to maintain.

## SPRING WS ENDPOINT RUNTIME

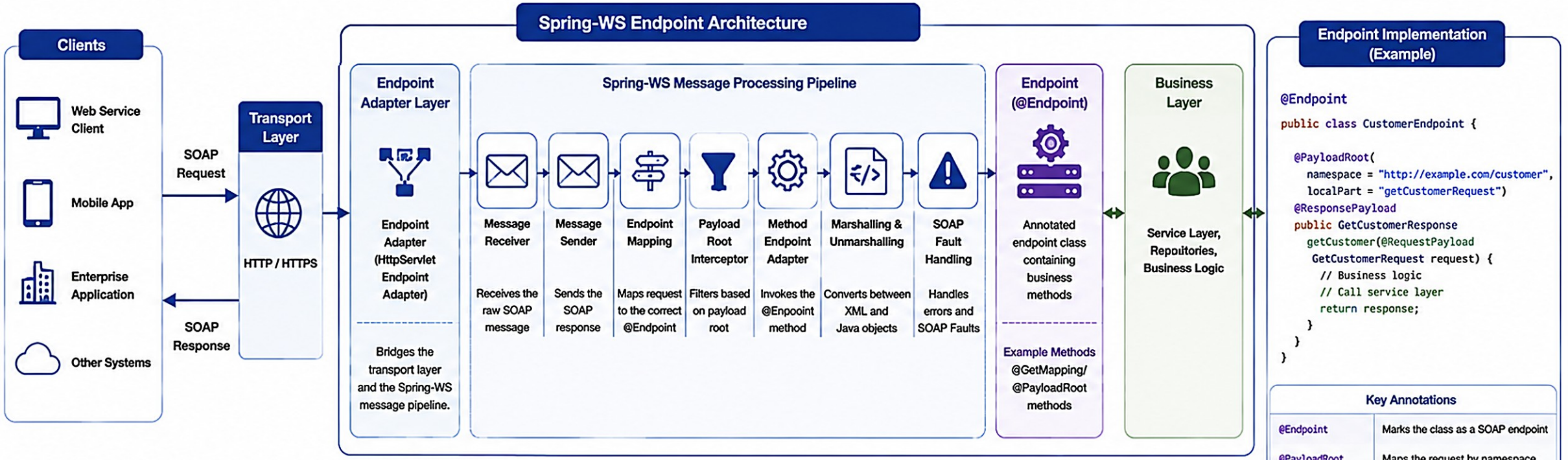

## BENEFITS OF THIS ARCHITECTURE

- Loose Coupling
- Scalability
- Maintainability
- Standards Compliant

**KEY TAKEAWAY** Endpoints are decoupled from clients and from the transport layer -- add or extend them independently as the contract grows.

# Endpoint Architecture

In Spring-WS, an Endpoint is the component that receives SOAP requests, processes them, and returns SOAP responses.




**Endpoint Implementation (Example)**

```java
@Endpoint
public class CustomerEndpoint {

  @PayloadRoot(
    namespace = "http://example.com/customer",
    localPart = "getCustomerRequest")
  @ResponsePayload
  public GetCustomerResponse
    getCustomer(@RequestPayload
     GetCustomerRequest request) {
      // Business logic
      // Call service layer
      return response;
    }
  }
}
```

**Key Annotations**

| | |
|---|---|
| `@Endpoint` | Marks the class as a SOAP endpoint |
| `@PayloadRoot` | Maps the request by namespace and local part |
| `@RequestPayload` | Binds the request payload to method parameter |
| `@ResponsePayload` | Binds the response object to SOAP body |

**Supporting Components**

- **Interceptors** (Logging, Security, Validation, etc.)
- **Security** (WS-Security, Authentication)
- **Validation** (Schema Validation, Custom Validation)
- **Fault Resolver** (Transforms exceptions into SOAP Faults)
- **Monitoring & Logging** (Tracing, Metrics, Audit Logs)

**Key Takeaways**

- Endpoints are the core entry point for handling SOAP requests in Spring-WS.
- The message pipeline processes, transforms, and routes SOAP messages.
- @Endpoint classes contain business methods mapped via @PayloadRoot.
- Interceptors, security, validation, and fault handling ensure enterprise-grade services.
- Clean separation: Transport → Pipeline → Endpoint → Business Layer.

# @ENDPOINT ANNOTATION (1 OF 2)

*The @Endpoint annotation marks a class as a SOAP endpoint -- Spring WS detects it and registers it as a web service endpoint.*

## WHAT IS @ENDPOINT?

**Class-level annotation —** declares that a class contains methods that handle incoming SOAP requests.

**Auto-registered —** allows Spring to scan and register the endpoint automatically.

**Method mapping —** methods within the class are mapped to SOAP operations using @PayloadRoot.

**Spring-native —** supports dependency injection and other Spring features.

## HOW @ENDPOINT WORKS


**KEY TAKEAWAY** @Endpoint promotes clean separation of concerns -- delegate business logic to service classes and keep endpoints stateless.

# @Endpoint Annotation in Spring WS

Marks a class as a Web Service endpoint to handle SOAP requests

The @Endpoint annotation identifies a class as a Spring Web Services endpoint.
Methods inside this class handle incoming SOAP requests.

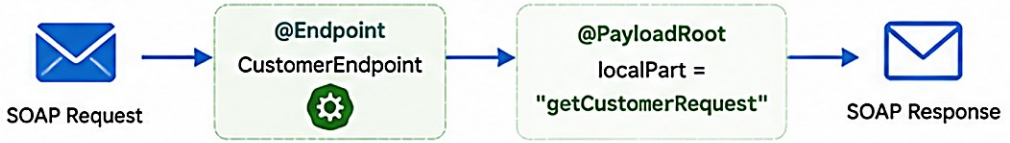
## 1. @Endpoint – Example

```
import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;
import javax.xml.transform.Source;

@Endpoint
public class CustomerEndpoint {
    private static final String NAMESPACE_URI = "http://example.com/customer";

    @PayloadRoot(namespace = NAMESPACE_URI, localPart = "getCustomerRequest")
    @ResponsePayload
    public GetCustomerResponse getCustomer(@RequestPayload GetCustomerRequest request) {
        // Business logic to fetch customer
        // Build and return response
        return buildResponse(request);
    }
}
```

## 2. Request Mapping with @Endpoint




Spring WS routes the SOAP request to the method whose @PayloadRoot namespace and localPart match the incoming request.

## 3. Key Points

- Identifies a class as a web service endpoint.
- Works with @PayloadRoot to map SOAP requests to methods.
- Methods return @ResponsePayload to send SOAP response.
- Promotes clean separation of concerns in service layer.
- Enables contract-first development with WSDL and XSD.

## 4. Endpoint Processing Flow

1. **SOAP Request** — Client sends SOAP request to the server.
2. **Message Dispatcher** — Spring WS receives the request and routes it.
3. **@Endpoint Detection** — Dispatcher finds the class annotated with @Endpoint.
4. **@PayloadRoot Matching** — Matches namespace and localPart to the method.
5. **Method Execution** — The matched method is invoked with the request.
6. **SOAP Response** — Response is marshalled and sent back to client.

## 5. Configuration

```
<sws:annotation-driven />  <!-- Enables annotation-based
                                endpoints -->
```

```
<bean class="org.springframewort.ws.transport.http.MessageDispatcherServlet">
  <property name="transformWsdlLocations" value="true"/>
</bean>
```

# @ENDPOINT ANNOTATION (2 OF 2)

*A working @Endpoint example, its annotation details, and best practices for endpoint classes.*

**CustomerEndpoint.java**

```java
@Endpoint
public class CustomerEndpoint {
    private static final String NS = "http://northstar.com/crm/customer";

    private final CustomerService customerService;

    @PayloadRoot(namespace = NS, localPart = "getCustomerRequest")
    @ResponsePayload
    public GetCustomerResponse getCustomer(
            @RequestPayload GetCustomerRequest request) {
        // delegate to the same service REST uses
        Customer c = customerService.get(request.getCustomerId());
        return CustomerSoapMapper.toSoap(c);
    }
}
```

## ANNOTATION DETAILS

| Field | Value |
|---|---|
| **Package** | org.springframework.ws.server.endpoint.annotation |
| **Target** | TYPE (Class) |
| **Retention** | RUNTIME |
| **Purpose** | Marks a class as a SOAP endpoint |

## BEST PRACTICES

- Use meaningful class names related to the domain.
- Delegate business logic to service classes -- never re-implement rules in the endpoint.
- Keep endpoints stateless whenever possible.

**KEY TAKEAWAY** CustomerEndpoint stays a thin protocol adapter -- it delegates so SOAP and REST never fork business rules.

# REQUEST MAPPING WITH @PAYLOADROOT

*@PayloadRoot maps an incoming SOAP request to a specific endpoint method based on the XML payload's namespace and local name.*

| Attribute | Meaning |
|---|---|
| **namespace** | Namespace URI of the root element in the request payload. |
| **localPart** | Name of the root element to be mapped (e.g. getCustomerRequest). |

**Example**

```
@PayloadRoot(namespace = NS, localPart = "getCustomerRequest")
@ResponsePayload
public GetCustomerResponse getCustomer(
    @RequestPayload GetCustomerRequest request) { ... }
```

## HOW REQUEST MAPPING WORKS

1. **SOAP Request**
2. **Message Dispatcher**
3. **@PayloadRoot Match**
4. **Handler Method**
5. **SOAP Response**

Simple & Declarative | Supports Contract-First | Works with WSDL/XSD | Keeps Methods Focused

**KEY TAKEAWAY** Use constants for the namespace URI -- four @PayloadRoot methods, one well-named method per operation.

# TRY IT YOURSELF — EXERCISE 2: SOAP OPERATION MAP

*Reinforces: mapping SOAP operations to shared CustomerService methods via @PayloadRoot you just saw.*

## STEP / WHAT TO DO

| Step | What To Do |
|---|---|
| **Goal** | Map four customer SOAP operations to shared CustomerService methods. |
| **File** | notes/soap-ops.md (in examples/module-24-exercises/). |
| **Map** | CreateCustomer -> create; GetCustomer -> get by id; UpdateCustomerStatus -> status transition; ListCustomers -> list/filter. |
| **Shared service rule** | Write that REST and SOAP must share CustomerService so business rules never fork. |
| **Deliverable** | Operation map + shared-service rule; list fixtures CUS-1001, CUS-1002, CUS-9999, correlation lab24-001. |
| **Reference** | exercises/exercise-02-operation-map.md |

## The Map You Are Writing (localPart -> CustomerService method)

```
@PayloadRoot(namespace = NS, localPart = "createCustomerRequest")        -> customerService.create(...)
@PayloadRoot(namespace = NS, localPart = "getCustomerRequest")           -> customerService.get(...)
@PayloadRoot(namespace = NS, localPart = "updateCustomerStatusRequest")  -> customerService.updateStatus(...)
@PayloadRoot(namespace = NS, localPart = "listCustomersRequest")         -> customerService.list(...)
```

**TRY IT NOW** Complete Exercise 2 before continuing -- see exercises/exercise-02-operation-map.md.

# TRY IT YOURSELF — EXERCISE 3: PAYLOADROOT SKELETON (STARTER)

*Reinforces: @Endpoint / @PayloadRoot delegation you just saw -- a fill-in-the-blank starter, not built from scratch.*

**notes/CustomerEndpointSketch.java (STARTER -- replace every blank)**

```
@_____
public class CustomerEndpoint {
    private final CustomerService service;

    public CustomerEndpoint(CustomerService service) {
        this.service = service;
    }

    @PayloadRoot(namespace = "http://northstar.example/customer",
                 localPart = "_____")
    @ResponsePayload
    public GetCustomerResponse get(
            @RequestPayload GetCustomerRequest request) {
        // TODO: call service.get(id) then map to JAXB response
        var customer = service._____(request.getCustomerId());
        return CustomerSoapMapper.toGetResponse(customer);
    }
}
```

## STEP / WHAT TO DO

| Step | What To Do |
|---|---|
| **Goal** | Complete a pseudocode endpoint skeleton that delegates to CustomerService. |
| **Starter** | TODO / fill-in-the-blank; replace every ____ and // TODO -- blanks do not compile. |
| **Hints** | Class annotation @Endpoint; localPart GetCustomerRequest; method service.get(...). |
| **Self-check** | No business rules live inside the endpoint beyond mapping/delegation. |
| **Deliverable** | All three blanks filled; delegation rule clear. |
| **Reference** | exercises/exercise-03-payloadroot-skeleton.md |

**TRY IT NOW** Complete Exercise 3 (fill in every blank) before continuing -- see exercises/exercise-03-payloadroot-skeleton.md.

# REQUEST PROCESSING WORKFLOW

*Spring WS processes a SOAP request through well-defined steps, from the transport layer to the endpoint method and back.*


## KEY COMPONENTS INVOLVED

**MessageDispatcher —** central component that routes messages to the correct endpoint.

**@PayloadRoot —** maps the XML root element (namespace + localPart) to the handler method.

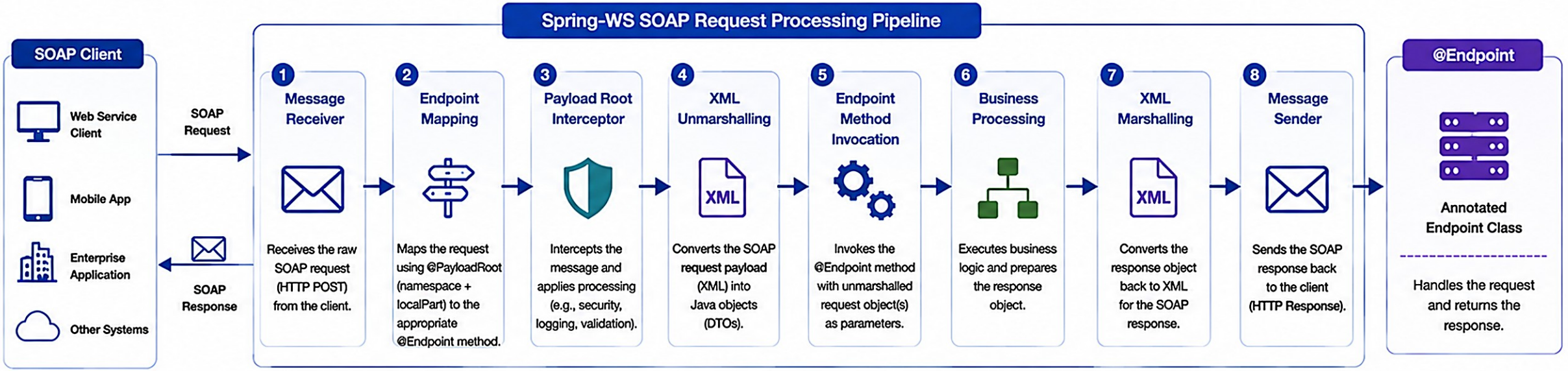
**Interceptors (Optional) —** used for logging, security, and validation at multiple points.

| Step | What Happens |
|---|---|
| **1-3. Receive & Dispatch** | SOAP request arrives; MessageReceiver builds a MessageContext; MessageDispatcher reads the payload. |
| **4-6. Map, Unmarshal & Invoke** | @PayloadRoot matches; payload unmarshalled to a Java object; handler method invoked. |
| **7-8. Marshal & Send** | Return value marshalled to XML; SOAP response sent back to the client. |

**KEY TAKEAWAY** A clear, predictable request flow with loose coupling between client and service -- and easy to extend with interceptors.

# SOAP Request Processing Workflow

How a SOAP request flows through Spring-WS from the client to the @Endpoint method.




## Annotation & Mapping

```
@Endpoint
public class CustomerEndpoint {

   @PayloadRoot(
       namespace = "http://example.com/customer",
       localPart = "getCustomerRequest"
   )
   @ResponsePayload
   public GetCustomerResponse getCustomer(
      @RequestPayload GetCustomerRequest request) {
      // business logic
      return response;
   }
}
```

- @PayloadRoot maps the incoming SOAP body (namespace + localPart) to the method.
- @RequestPayload binds the request XML to Java object.
- @ResponsePayload marks the object to be returned in the SOAP response.

## Key Components Involved

- **MessageReceiver** — Receives SOAP request from transport layer.
- **EndpointMapping** — Resolves request to @Endpoint method.
- **Interceptors** — Cross-cutting concerns (security, logging, validation).
- **Marshalling / Unmarshalling** — Converts between XML and Java objects.
- **EndpointAdapter** — Invokes the target @Endpoint method.
- **Business Logic** — Performs the application logic.
- **MessageSender** — Sends SOAP response back to the client.
- **FaultResolver** — Handles exceptions and creates SOAP Fault.

## Flow Summary

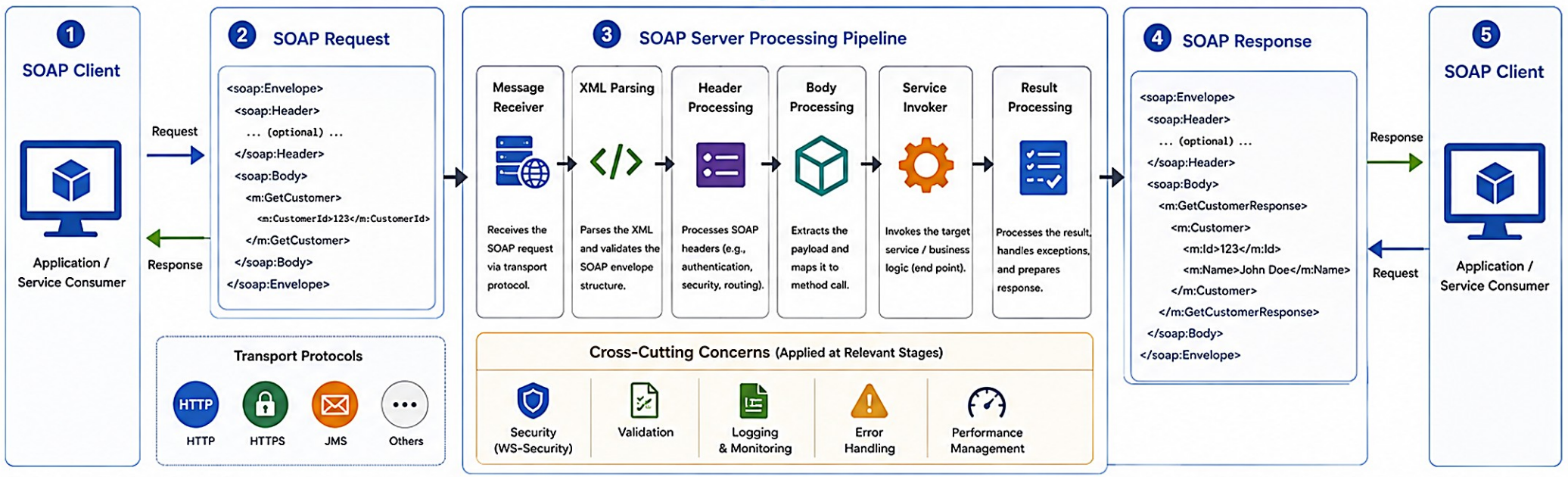
1. Client sends SOAP request (HTTP POST)
2. Spring-WS receives the message
3. Request is mapped to @Endpoint method
4. XML is unmarshalled to Java objects
5. @Endpoint method is invoked
6. Business logic processes the request
7. Response is marshalled back to XML
8. SOAP response is returned to the client

## Key Takeaways

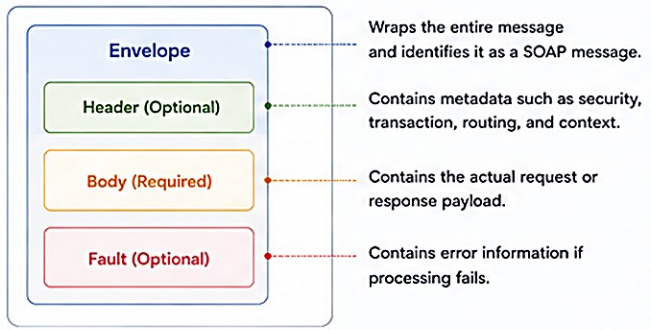
- Spring-WS follows a structured pipeline to process SOAP requests.
- Interceptors handle cross-cutting concerns consistently.
- Marshalling/Unmarshalling enables clean XML ↔ Java object conversion.
- Clear mapping with @PayloadRoot ensures loose coupling and maintainability.

# SOAP Request Processing Architecture

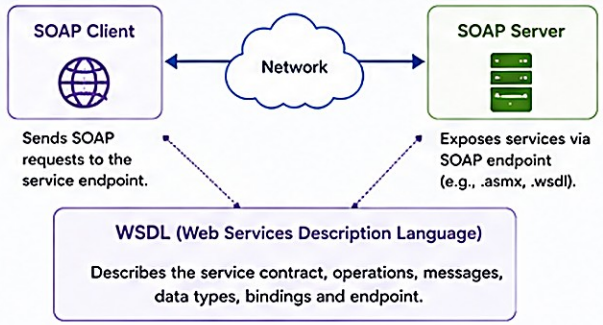
End-to-end flow of a SOAP request from client to server and back




## SOAP Message Structure




## Typical SOAP Endpoints (Deployment Style)




## Key Benefits of SOAP Architecture

- **Standard & Interoperable** — W3C standard ensures platform and language independence.
- **Secure** — Supports WS-Security, encryption, digital signatures.
- **Contract-First** — WSDL defines a strict contract between client and server.
- **Reliable** — Built-in error handling with SOAP Fault and robust messaging.
- **Enterprise Ready** — Ideal for large-scale, mission-critical enterprise systems.

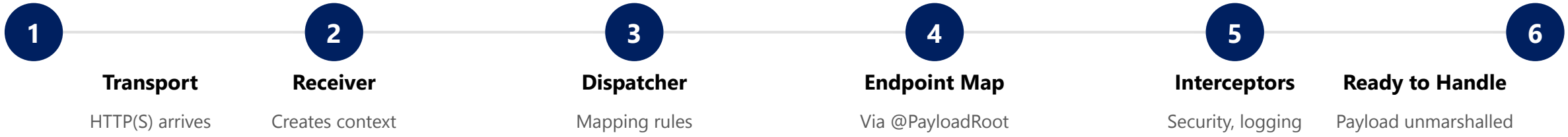
SOAP + XML + Standards = Interoperable, Secure, and Reliable Enterprise Integration

Request Flow → Response Flow

SOAP is transport neutral and can work over HTTP, HTTPS, JMS and other protocols.

# SOAP REQUEST LIFECYCLE

*The journey of a request from the client to the Spring WS endpoint, until it is ready for the handler method.*




## WHAT HAPPENS IN THIS LIFECYCLE?

- The HTTP request is accepted and the SOAP envelope is parsed.
- The correct endpoint is resolved using @PayloadRoot mapping information.
- Cross-cutting concerns (security, logging, validation) are applied via interceptors.
- The payload is converted from XML to Java objects before the handler runs.

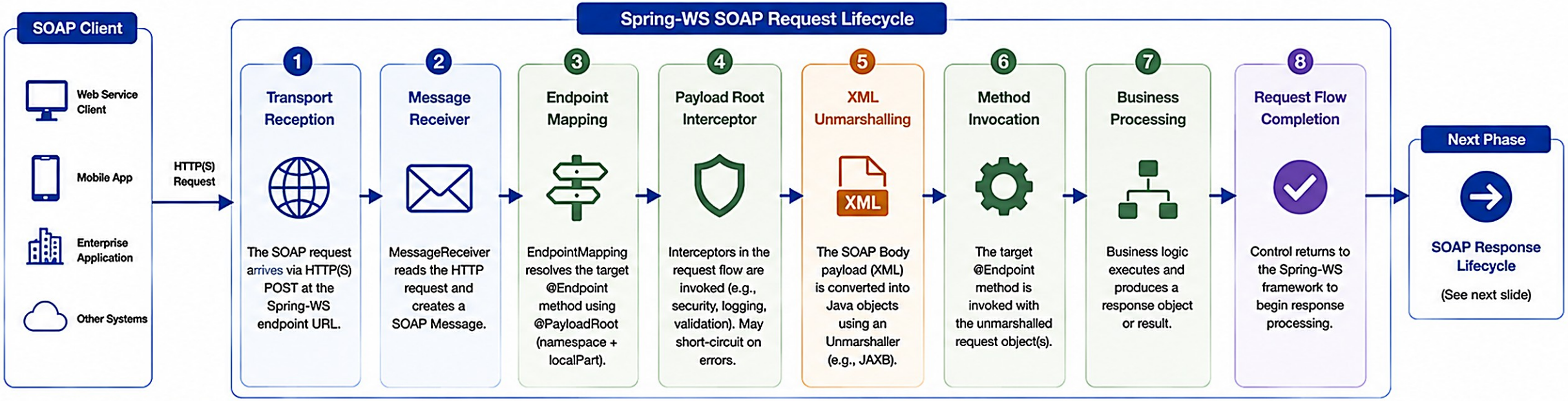
**KEY TAKEAWAY** These steps occur for every incoming SOAP request before the business logic is executed.

# SOAP Request Lifecycle

How a SOAP request is processed inside Spring-WS from transport entry to business method invocation.

**Legend**

Transport | Processing | Marshalling | Application




## Participants

| | |
|---|---|
| Client | Sends the SOAP request |
| Spring-WS Framework | Handles transport, routing, and processing |
| @Endpoint | Contains business methods |
| Backend Systems | Used by business logic if needed |

## Key Components in the Pipeline

**MessageReceiver** — Creates SOAP Message from transport request.

**EndpointMapping** — Maps request to @Endpoint method using @PayloadRoot.

**Interceptors** — Cross-cutting concerns: security, logging, validation, metrics.

**Unmarshaller** — Converts XML payload to Java objects (JAXB / XmlBeans).

**@Endpoint Method** — Business method invoked with request object(s).

**Business Logic** — Executes application logic and builds response.

## Lifecycle Summary

1. Transport layer receives the SOAP request.
2. SOAP Message is created.
3. Target @Endpoint method is resolved.
4. Interceptors in the request flow execute.
5. XML payload is unmarshalled to Java objects.
6. @Endpoint method is invoked.
7. Business logic processes the request.
8. Control returns to framework for response phase.

## Key Takeaways

- The request lifecycle transforms a SOAP message into a Java method invocation.
- Interceptors enable security, validation, logging, and other cross-cutting concerns.
- Unmarshalling converts XML payloads into strongly typed Java objects.
- Clear pipeline stages ensure maintainability, extensibility, and observability.

# SOAP RESPONSE LIFECYCLE

*The journey of a response from the endpoint method back to the client, until it is received and processed.*

1. **Endpoint Done** — Logic completes
2. **Marshalled** — Java object -> XML
3. **Message Built** — SOAP Body assembled
4. **Interceptors Run** — Headers, auditing
5. **Message Sent** — Sent via HTTP(S)
6. **Client Receives** — Client parses XML

## WHAT IF SOMETHING GOES WRONG?

If an exception is thrown in the endpoint method, an appropriate SOAP Fault is created during this lifecycle and returned to the client instead of a normal response -- see the next section.

**Consistent Handling** | **Supports Cross-Cutting Concerns** | **Easier Troubleshooting**

**KEY TAKEAWAY** The response lifecycle mirrors the request lifecycle -- and it's where a thrown exception becomes a SOAP Fault.

# SOAP Response Lifecycle

How a SOAP response is generated in Spring-WS and returned to the client.

**Legend**

- Application / Business
- Marshalling
- Processing
- Transport

## SOAP Client

- Web Service Client
- Mobile App
- Enterprise Application
- Other Systems

HTTP(S) Response

## Spring-WS SOAP Response Lifecycle

**8 — Client Reception**
The client receives the SOAP response and processes the result.

**7 — Transport Transmission**
The response is sent via HTTP(S) back to the client.

**6 — Message Sender**
The MessageSender writes the SOAP response to the transport layer.

**5 — Endpoint Adapter**
The response message is given to the Endpoint Adapter for further handling.

**4 — Message Enrichment**
Interceptors may add headers, security tokens, logging info, correlation IDs, etc.

**3 — Message Creation**
The XML is placed in a SOAP Body and a SOAP Message is created.

**2 — XML Marshalling**
The response object is marshalled into XML by the Marshaller (e.g., JAXB).

**1 — Business Logic Result**
@Endpoint method finishes processing and returns a result (POJO / DTO / Response Object).

## Next Phase

**Lifecycle Complete**

Response delivered successfully to the client.

## Key Components Involved

**@Endpoint Method**
Executes business logic and returns a response object.

**Marshaller**
Converts Java objects to XML payload (JAXB / XmlBeans).

**SOAP Message**
Holds the SOAP Envelope, Headers (optional) and Body.

**Interceptors**
Enrich, secure, log, or modify the response message.

**Endpoint Adapter**
Routes the response through the Spring-WS framework.

**MessageSender**
Handles writing the response to the transport.

**Transport Layer**
Sends the SOAP response over HTTP(S).

## Lifecycle Summary (Response Path)

1. @Endpoint method returns a response object.
2. Response object is marshalled to XML.
3. SOAP Message is created with XML in the Body.
4. Interceptors may enrich or secure the message.
5. Endpoint Adapter handles the response.
6. MessageSender writes the response to transport.
7. Transport layer sends the response to the client.
8. Client receives and processes the response.

## Typical SOAP Response Message

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Header>
        <!-- Optional: WS-Security, Tracking, etc. -->
    </soap:Header>
    <soap:Body>
        <ns:getCustomerResponse xmlns:ns="http://example.com/customer">
            <ns:status>SUCCESS</ns:status>
            <ns:customerId>12345</ns:customerId>
            <ns:name>John Doe</ns:name>
        </ns:getCustomerResponse>
    </soap:Body>
</soap:Envelope>
```

Headers (Optional)

Body (Payload)

## Key Takeaways

- The response travels back through the same pipeline in reverse.
- Marshalling converts Java objects to XML for the SOAP Body.
- Interceptors ensure security, logging and consistency.
- A well-structured lifecycle ensures reliable and standardized responses.

# XML MARSHALLING

*XML Marshalling is the process of converting Java objects into XML so they can be included in a SOAP message.*

1. **Java Object (POJO)**
2. **Marshaller**
3. **XML Output**
4. **SOAP Message**

**Java Object**

```
Customer customer = new Customer();
customer.setId("CUS-1001");
customer.setName("Amina Khan");
customer.setEmail("amina@northstar.com");
```

**Marshalled XML (Output)**

```
<customer>
  <id>CUS-1001</id>
  <name>Amina Khan</name>
  <email>amina@northstar.com</email>
</customer>
```

## KEY POINTS

- Uses a Marshaller (typically JAXB / Jakarta XML Binding) to convert Java objects to XML.
- Follows the structure defined in the XSD or JAXB annotations -- always validate against the schema first.

**KEY TAKEAWAY** Marshalling ensures your Java data is correctly transformed into XML that follows the SOAP contract.

# Marshalling in SOAP (Spring-WS)

Marshalling is the process of converting Java objects into XML so they can be included in the SOAP message request or response.

## Marshalling in the SOAP Message Flow

**1. Java Objects**
Application creates Java objects

Customer
id: 101
name: John Doe
email: john@example.com

Java Object / POJO

**2. Marshalling**
Marshaller converts the Java object into XML

Marshaller
(e.g., JAXB2Marshaller, Jackson2XmlMarshaller, AxiomMarshaller)

**3. XML Creation**
XML representation of the object is created

**4. SOAP Message Construction**
XML is placed in the SOAP Body

**5. Transport**
The SOAP message is sent to the endpoint

HTTP / HTTPS

**6. Web Service Endpoint**
Receives and processes the SOAP message

Spring-WS Endpoint

## How Marshalling Works in Spring-WS

1. Spring-WS selects a Marshaller based on configuration.
2. The Marshaller uses metadata (annotations or mappings) to understand the object structure.
3. It converts Java objects into XML according to the schema or mapping rules.
4. The XML is returned to the framework.
5. The framework places the XML into the SOAP Body.
6. The SOAP message is sent over HTTP/HTTPS.

## Common Marshallers in Spring-WS

**JAXB2Marshaller**
- Uses JAXB (Jakarta XML Binding)
- Annotations (@XmlElement, @XmlRootElement)
- Schema generation support
- Most commonly used

**Jackson2XmlMarshaller**
- Uses Jackson XML dataformat
- JSON and XML mapping with Jackson
- Flexible and lightweight

**AxiomMarshaller**
- Uses Apache Axiom
- Pull-based XML building
- Good for large XML documents

**CastorMarshaller**
- Uses Castor framework
- Mapping via XML or annotations
- Schema generation support

## Example

**Java Object (POJO)**

```
public class Customer {
    private Long id;
    private String name;
    private String email;
    // getters and setters
}
```

**Marshalled XML**

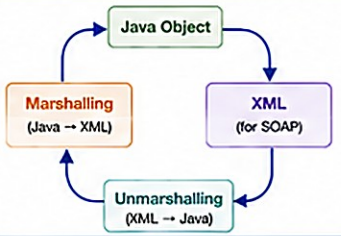
```
<ns:Customer xmlns:ns="http://example.com/schema">
    <ns:id>101</ns:id>
    <ns:name>John Doe</ns:name>
    <ns:email>john@example.com</ns:email>
</ns:Customer>
```

**SOAP Message (Request) – With Marshalled XML**

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:ns="http://example.com/schema">
 <soapenv:Header/>
 <soapenv:Body>
   <ns:getCustomerRequest>
     <ns:Customer>
       <ns:id>101</ns:id>
       <ns:name>John Doe</ns:name>
       <ns:email>john@example.com</ns:email>
     </ns:Customer>
   </ns:getCustomerRequest>
 </soapenv:Body>
</soapenv:Envelope>
```

## Key Points

- Marshalling converts Java objects → XML.
- The Marshaller uses object metadata (annotations or mapping files).
- The resulting XML is inserted into the SOAP Body.
- Proper marshalling ensures correct message structure and schema compliance.
- Unmarshalling is the reverse process: XML → Java objects.




## Key Takeaways

- Marshalling is essential for creating SOAP messages from Java objects.
- Spring-WS provides multiple marshallers to choose from.
- Using JAXB2Marshaller with annotations is the most widely used approach.
- Correct marshalling ensures interoperability, validation, and standards compliance.
- Marshalling happens automatically in the request/response pipeline based on configuration.

# XML UNMARSHALLING

*XML Unmarshalling converts XML from a SOAP request into Java objects so the application can process the data.*


**SOAP Request XML**

```
<getCustomerRequest>
  <customerId>CUS-1001</customerId>
</getCustomerRequest>
```

**Java Object (Output)**

```
GetCustomerRequest request =
    new GetCustomerRequest();
request.setCustomerId("CUS-1001");
// passed to the @RequestPayload parameter
```

## KEY POINTS

- Extracts data from the SOAP request and validates the XML structure using XSD (if configured).
- Populates strongly-typed Java objects (POJOs) for business processing -- no manual XML parsing.

**KEY TAKEAWAY** Unmarshalling transforms XML into usable Java objects for clean, type-safe business logic.

# Unmarshalling in SOAP (Spring-WS)

Unmarshalling is the process of converting XML from a SOAP message into Java objects so that the application can process the data.

## Unmarshalling in the SOAP Response Flow (Server Side)

**1 SOAP Message Received**
The SOAP response (XML) is received at the Spring-WS endpoint.
XML over HTTP/HTTPS

**2 Message Receiver**
MessageReceiver reads the incoming SOAP message.

**3 XML Extraction**
The SOAP Body is extracted from the message.
SOAP Body (XML)

**4 Unmarshalling**
The Unmarshaller converts the XML into Java objects based on the schema (XSD) or binding.

**5 Java Object Created**
Java objects (POJOs) are created and populated with values from the XML.

**6 Object Passed to Endpoint**
The Java object is passed to the @Endpoint method for business processing.

## How Unmarshalling Works in Spring-WS

1. Spring-WS reads the SOAP message from the transport.
2. The SOAP Body (XML payload) is extracted.
3. The Unmarshaller converts XML into Java objects.
4. Schema mappings help convert XML elements to fields.
5. A fully populated Java object is returned.
6. The object is passed to the @Endpoint method.

## Common Unmarshallers in Spring-WS

**JAXB2Marshaller**
- Uses JAXB (Jakarta XML Binding)
- Binding based on annotations or generated classes
- Most commonly used

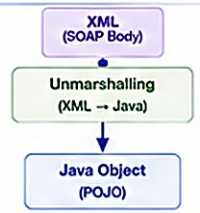
**CastorMarshaller**
- Uses Castor framework
- XML mapping files or annotations
- Schema driven

**XStreanMarshaller**
- Uses XStream library
- Simple to configure
- Good for simple XML structures

**JiBXMarshaller**
- Uses JiBX binding
- High performance
- Binding files required

## Key Points

- Unmarshalling converts XML → Java objects.
- It is the reverse of Marshalling.
- The Unmarshaller uses schema or bindings to map XML to Java.
- Correct schema and mappings ensure accurate object creation.
- Unmarshalling occurs before business logic execution in @Endpoint methods.




## Example: Incoming SOAP Response (XML)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:ns="http://example.com/schema">
   <soapenv:Header/>
   <soapenv:Body>
     <ns:getCustomerResponse>
       <ns:status>SUCCESS</ns:status>
       <ns:customer>
         <ns:id>101</ns:id>
         <ns:name>John Doe</ns:name>
         <ns:email>john@example.com</ns:email>
       </ns:customer>
     </ns:getCustomerResponse>
   </soapenv:Body>
</soapenv:Envelope>
```

## Java Object After Unmarshalling (POJO)

```
public class CustomerResponse {
   private String status;
   private Customer customer;
   // getters and setters
}
public class Customer {
   private Long id;
   private String name;
   private String email;
   // getters and setters
}
```

## Unmarshalling Configuration in Spring-WS

```
<bean id="unmarshaller" class="org.springframework.oxm.jaxb.Jaxb2Marshaller">
   <property name="contextPath" value="com.example.schema"/>
</bean>
```

The Unmarshaller uses the schema (XSD) or JAXB bindings to convert XML into Java objects.

## Key Takeaways

- Unmarshalling is essential for reading and processing SOAP responses.
- It transforms XML payloads into Java objects automatically.
- Schema or binding files define how XML maps to Java.
- Accurate mappings improve reliability, maintainability, and reduce errors.
- Spring-WS provides multiple Unmarshallers to choose from based on needs.

# SOAP FAULT STRUCTURE AND ERROR HANDLING

*When an endpoint throws an exception, Spring WS converts it into a structured SOAP Fault instead of a raw stack trace.*

## ANATOMY OF A SOAP FAULT

**faultcode —** a qualified code such as Client or Server, identifying who is at fault.

**faultstring —** a short, human-readable description of the error (never a stack trace).

**faultactor —** identifies which node in the message path caused the fault (optional).

**detail —** optional structured, application-specific error information.

| Fault Code | Meaning |
|---|---|
| **Client** | Caller sent a bad/invalid request (e.g. not-found, duplicate). |
| **Server** | Something failed on the server side; generic message, no internals. |
| **MustUnderstand** | A required SOAP header element was not understood. |
| **VersionMismatch** | The SOAP envelope version is not supported. |

### SoapFaultMappingExceptionResolver

```
@Bean
SoapFaultMappingExceptionResolver exceptionResolver() {
  var resolver = new SoapFaultMappingExceptionResolver();
  var mappings = new Properties();
  mappings.setProperty(
    "com.northstar.crm.exception.CustomerNotFoundException",
    "CLIENT,Customer not found");
  resolver.setExceptionMappings(mappings);
  return resolver;
}
```

### Fault Response (CUS-9999, not found)

```
<SOAP-ENV:Fault>
  <faultcode>SOAP-ENV:Client</faultcode>
  <faultstring>Customer not found</faultstring>
</SOAP-ENV:Fault>
```

Never let faultstring or detail leak stack traces or secrets.

**KEY TAKEAWAY** Map business exceptions to CLIENT faults with clear messages; default everything else to SERVER -- no stack traces, ever.

# TRY IT YOURSELF — EXERCISE 4: SOAP FAULT VS. REST ERROR

*Reinforces: mapping business exceptions to SOAP faults without leaking stacks you just covered.*

## STEP / WHAT TO DO

| Step | What To Do |
|---|---|
| **Goal** | Document how business exceptions become SOAP faults without leaking stacks. |
| **File** | notes/fault-vs-rest.md (in examples/module-24-exercises/). |
| **Contrast table** | Columns Concern \| SOAP \| REST; rows: not-found, validation, missing UsernameToken. |
| **Answer sketch** | Not-found -> SOAP fault vs HTTP 404 JSON; missing token -> security fault vs 401 on REST. |
| **Deliverable** | Contrast table + no-stack-trace rule; note Lab 16 exceptions feed Lab 24 fault mapping. |
| **Reference** | exercises/exercise-04-fault-vs-rest.md |

## Same Failure, Two Protocols (what your contrast table is documenting)

```
SOAP:  <SOAP-ENV:Fault><faultcode>SOAP-ENV:Client</faultcode><faultstring>Customer not found</faultstring></SOAP-ENV:Fault>
REST:  HTTP 404 { "error": "NOT_FOUND", "message": "Customer not found" }
Rule:  faultstring/detail and REST error bodies never include stack traces, class names, or secrets.
```

**TRY IT NOW** Complete Exercise 4 before continuing -- see exercises/exercise-04-fault-vs-rest.md.

# SOAP SECURITY OVERVIEW

*SOAP provides several standards and mechanisms (WS-Security) to ensure confidentiality, integrity, authentication, and non-repudiation.*

## GOALS OF SOAP SECURITY

| Goal | What It Ensures |
|---|---|
| **Confidentiality** | Message content is not accessible to unauthorized parties. |
| **Integrity** | Message content is not altered in transit. |
| **Authentication** | Verifies the identity of the sender and/or receiver. |
| **Non-Repudiation** | Provides cryptographic proof of origin and prevents denial of actions. |

## WS-SECURITY STANDARD

WS-Security is an OASIS standard that extends SOAP to add security tokens and related processing to SOAP messages -- independent of the transport layer.

## COMMON SECURITY TOKENS

- **UsernameToken**
- **SAML Token**
- **X.509 Certificate**
- **BinarySecurityToken**

Always use HTTPS for transport security in addition to WS-Security -- message-level security and transport security solve different problems.

**KEY TAKEAWAY** SOAP security combines HTTPS transport with message-level WS-Security so identity survives intermediaries.

# SOAP Security Overview

Securing SOAP Web Services for Confidentiality, Integrity, and Authentication

SOAP Security (WS-Security) is a standard specification that adds security to SOAP messages using XML-based security tokens and headers.

## 1. OVERVIEW

SOAP Security (WS-Security) provides a standard way to secure SOAP messages at the message level.

- Ensures Confidentiality
- Ensures Integrity
- Provides Authentication
- Prevents Replay Attacks
- Works with existing SOAP and XML infrastructure

## 2. SECURITY BENEFITS

**Data Confidentiality**
Protects data from being read by unauthorized parties.

**Data Integrity**
Ensures messages are not altered in transit.

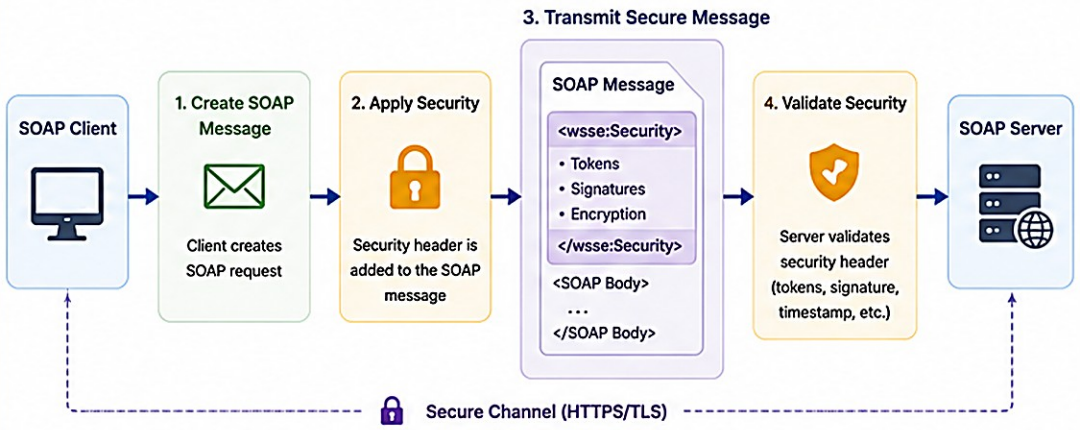
**Authentication**
Verifies the identity of the sender.

**Non-Repudiation**
Provides proof of message origin.

## 3. WS-SECURITY STANDARDS

- ☑ OASIS WS-Security 1.1 (2006)
- ☑ OASIS WS-Security 1.1.1 (UsernameToken Profile)
- ☑ OASIS WS-Security SAML Token Profile
- ☑ OASIS WS-Security X.509 Token Profile

## 4. HOW WS-SECURITY WORKS




## 5. WS-SECURITY MESSAGE ELEMENTS

SOAP Message Structure

```
<soap:Envelope>
  <soap:Header>
    <wsse:Security>
      <!-- Security Elements -->
    </wsse:Security>
  </soap:Header>
  <soap:Body>
      <!-- Business Payload -->
  </soap:Body>
</soap:Envelope>
```

| | |
|---|---|
| Security Header (wsse:Security) | Container for all security tokens and elements. |
| Security Tokens | Provide authentication information (e.g., UsernameToken, X.509, SAML token). |
| Digital Signature | Ensures message integrity and non-repudiation. |
| Encryption | Ensures confidentiality of message content. |
| Timestamp | Prevents replay attacks by validating message time. |

## 6. COMMON SECURITY FEATURES

**UsernameToken**
Uses username/password credentials for authentication.

**X.509 Certificate Token**
Uses digital certificates for strong authentication.

**SAML Token**
Supports enterprise single sign-on (SSO) and federation.

**Message Encryption**
Encrypts SOAP body, headers, or specific elements.

**Message Signature**
Digitally signs whole message or specific parts.

**Timestamp**
Adds timestamp to ensure message freshness.

**Non-Repudiation**
Provides proof of message origin and integrity.

## 7. BEST PRACTICES

- Always use HTTPS/TLS for transport security.
- Use strong tokens (X.509 or SAML) for production systems.
- Validate timestamps to prevent replay attacks.
- Encrypt sensitive data inside the SOAP body.
- Sign important parts of the message.
- Keep security libraries and certificates up to date.

# WS-SECURITY CONCEPTS

*WS-Security extends SOAP messages by adding security information in the SOAP Header -- most commonly a UsernameToken.*

**wsse:Security Header (UsernameToken)**

```
<soapenv:Header>
  <wsse:Security xmlns:wsse="...oasis-200401-wss-wssecurity-secext-1.0.xsd">
    <wsse:UsernameToken>
      <wsse:Username>crm-partner</wsse:Username>
      <wsse:Password Type="...PasswordText">
        lab24-shared-secret
      </wsse:Password>
    </wsse:UsernameToken>
  </wsse:Security>
</soapenv:Header>
```

## CORE WS-SECURITY CONCEPTS

**Security Tokens —** carry authentication info about the sender (UsernameToken, SAML, X.509).

**Digital Signature —** ensures integrity and authenticity using XML Signature.

**Encryption —** protects message content using XML Encryption.

**Timestamp —** prevents replay attacks by adding creation time and expiration.

## HOW A WSS4J USERNAMETOKEN INTERCEPTOR WORKS

1. **SOAP Msg + Header**
2. **Wss4j Interceptor**
3. **Validates Token**
4. **Reject or Forward**
5. **Endpoint Runs**

**KEY TAKEAWAY** Plaintext PasswordText UsernameToken is lab-only -- production needs TLS plus PasswordDigest (or better) and rotated secrets.

# WS-Security in SOAP Web Services

WS-Security is a standard (OASIS) that adds security to SOAP messages.
It defines how to provide authentication, integrity, confidentiality and access control.

## Security Goals

- **Authentication** — Verify the identity of the sender
- **Confidentiality** — Protect the message from eavesdropping
- **Integrity** — Ensure the message is not tampered
- **Access Control** — Ensure only authorized parties can act

## How WS-Security Works (Message Flow)

1. **Client Creates SOAP Request** — The client application creates a SOAP request.
2. **Apply Security (WS-Security)** — Security tokens and policies are applied to the SOAP Header.
3. **Send Secured SOAP Message** — The SOAP message with WS-Security is sent over HTTP/HTTPS.
4. **Server Interception** — The server receives the message and intercepts it.
5. **Security Processing** — WS-Security engine validates tokens, checks integrity and authenticates the sender.
6. **Application Processing** — If the message is valid, it is passed to the endpoint for business processing.
7. **SOAP Response with Security** — The response is secured and sent back to the client.
8. **Client Verification** — The client verifies the response before processing it.

## WS-Security Header Structure

```
<soapenv:Header>
  <wsse:Security soapenv:mustUnderstand="1"
    xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/
    oasis-200401-wss-wsecurity-secext-1.0.xsd">
     <!-- Security Token -->
     <wsse:UsernameToken>
       <wsse:Username>john.doe</wsse:Username>
       <wsse:Password Type="....#PasswordDigest">...</wsse:Password>
       <wsse:Nonce>...</wsse:Nonce>
       <wsu:Created>2024-05-20T10:15:30Z</wsu:Created>
     </wsse:UsernameToken>
     <!-- Digital Signature -->
     <ds:Signature xmlns:ds="http://www.w3.org/2000/09/xmldsig#">...
     </ds:Signature>
     <!-- Encryption (optional) -->
     <xenc:EncryptedKey>...</xenc:EncryptedKey>
  </wsse:Security>
</soapenv:Header>
<soapenv:Body> ... </soapenv:Body>
```

## Common WS-Security Elements

### Authentication
- UsernameToken (Password / Digest)
- X.509 Certificate
- SAML Token
- Kerberos Token

### Integrity
- XML Digital Signature
- Ensures message is not changed
- Signed parts: Body, Timestamp, etc.

### Confidentiality
- XML Encryption
- Encrypts parts of the SOAP message
- Protects sensitive data in transit

### Non-Repudiation
- Digital Signature
- Prevents sender from denying the message
- Protects proof of origin and integrity

### Timestamp
- Prevents replay attacks
- <wsu:Timestamp> contains Created and Expires

## Security Token Examples

- **UsernameToken** — Used for simple authentication (user/password).
- **X.509 Certificate** — Uses digital certificates for strong authentication.
- **SAML Token** — Carries assertions about user identity and roles.
- **Kerberos Token** — Enterprise authentication using Kerberos tickets.

## WS-Security Policy (Example)

```
<wsp:Policy>
  <sp:AsymmetricBinding>
    <wsp:Policy>
      <sp:InitiatorToken>
        <wsp:Policy>
          <sp:X509Token sp:IncludeToken="http://schemas.xmlsoap.org/
            ws/2005/07/securitypolicy/IncludeToken/AlwaysToRecipient"/>
        </wsp:Policy>
      </sp:InitiatorToken>
      <sp:RequireSignatureConfirmation/>
      <sp:Layout>
        <sp:LaxTimestampFirst/>
      </sp:Layout>
    </wsp:Policy>
  </sp:AsymmetricBinding>
</wsp:Policy>
```

- Require X.509 certificate
- Require signature confirmation
- Include timestamp to prevent replay

## WS-Security in Action (End-to-End View)

Client → Add Security (Header) → SOAP Message (Encrypted/Signed) → Transport Layer (HTTPS) → Server → Validate Security (Tokens, Signature) → Endpoint Business Logic → SOAP Response (Secured) → Client Verification

## Best Practices

- ☑ Always use HTTPS for transport security.
- ☑ Sign and encrypt sensitive SOAP messages.
- ☑ Use timestamps to prevent replay attacks.
- ☑ Validate all incoming tokens and signatures.
- ☑ Follow the principle of least privilege.

## Key Takeaways

- WS-Security secures SOAP messages at the message level.
- It provides standards for authentication, integrity, confidentiality, and access control.
- It works independently of transport security (but HTTPS is recommended).
- It is widely used in enterprise environments and with legacy systems.

# TRY IT YOURSELF — EXERCISE 5: USERNAMETOKEN PLAN

*Reinforces: WSS4J UsernameToken validation you just saw -- planning the evidence, not implementing JWT.*

## STEP / WHAT TO DO

| Step | What To Do |
|---|---|
| **Goal** | Outline UsernameToken evidence without implementing JWT. |
| **File** | notes/usernametoken-plan.md (in examples/module-24-exercises/). |
| **Happy path** | Secured GetCustomer for CUS-1001 (crm-partner / lab24-shared-secret) succeeds. |
| **Failure path** | Missing/invalid token produces a distinct security fault from a not-found fault. |
| **Deliverable** | Secret hygiene rule (never real prod passwords) stated; Bearer JWT explicitly deferred to Lab 28. |
| **Reference** | exercises/exercise-05-usernametoken-plan.md |

## The Two Scenarios You Are Planning For

```
requests/get-customer-secured.xml   -> crm-partner / lab24-shared-secret UsernameToken -> Wss4j validates -> CustomerEndpoint runs
requests/get-customer.xml           -> no Security header                               -> Wss4j rejects  -> SOAP security fault (never reaches endpoint)
```

**TRY IT NOW** Complete Exercise 5 before continuing -- see exercises/exercise-05-usernametoken-plan.md.

# INTEGRATING SOAP SERVICES (1 OF 2)

*Integrating SOAP services means consuming external SOAP web services in your application using standard SOAP messages.*

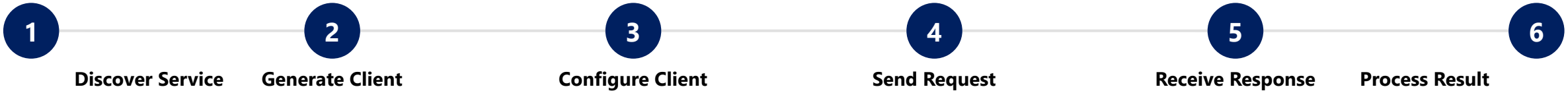
## WAYS TO INTEGRATE SOAP SERVICES

**1. WSDL-First Approach —** use the service's published WSDL to generate client stubs and strongly typed Java classes.

**2. Contract-First Approach —** define your own contract (WSDL/XSD) first, then generate client and server artifacts.

**3. Dynamic / Template Approach —** use WebServiceTemplate for dynamic requests without generating stubs.

## INTEGRATION FLOW (SOAP CLIENT)




**KEY TAKEAWAY** Integrating SOAP services ensures seamless, standards-compliant communication with external systems.

# Integrating SOAP Services

Connect and consume external SOAP Web Services in enterprise applications

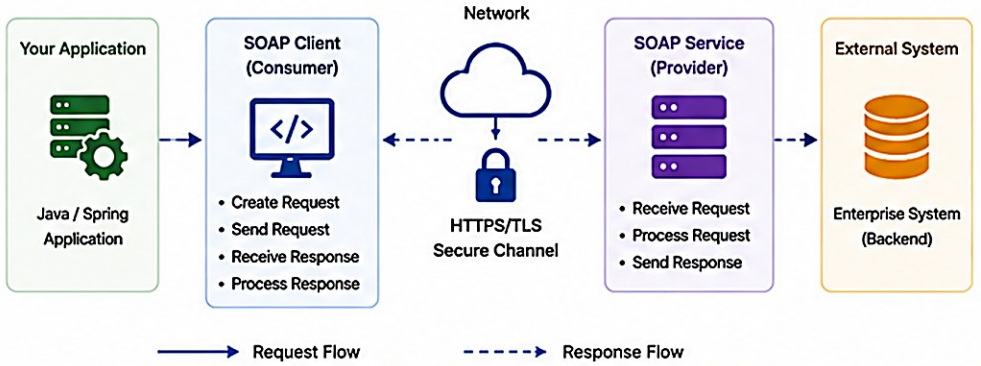
SOAP integration enables systems to communicate securely and reliably across organizational boundaries.

## 1. OVERVIEW

SOAP integration allows your application to communicate with external systems exposing SOAP web services.

- Standards-based (XML, WSDL)
- Platform independent
- Enterprise-grade security
- Reliable and transactional
- Widely used in legacy and enterprise systems

## 2. SOAP INTEGRATION ARCHITECTURE




## 3. INTEGRATION APPROACHES

**Code-First (Client-First)**
- Generate client from WSDL
- Use Spring WS / WebServiceTemplate
- Quick to consume existing services

**Contract-First**
- Define contract (WSDL/XSD) first
- Generate server stubs
- Build service implementation

**Service to Service Integration**
- System-to-system communication
- Orchestration and routing
- Error handling and monitoring

## 4. KEY CONSIDERATIONS

**Security**
Include-Security, SSL/TLS, and proper authentication.

**WSDL & Versioning**
Ensure compatibility and manage service versions.

**Performance**
Optimize with caching, connection pooling, and async calls.

**Error Handling**
Handle SOAP faults and network errors gracefully.

**Monitoring & Logging**
Track requests, responses, and integration metrics.

## 5. EXAMPLE: CONSUMING A SOAP SERVICE IN SPRING

1. **Add Dependency** — Include Spring WS in your project (Maven/Gradle).
2. **Load WSDL** — Provide the WSDL URL or file to generate the client.
3. **Create Client** — Use WebServiceTemplate or generated client stubs.
4. **Send Request** — Marshal request object to SOAP message and send.
5. **Process Response** — Unmarshal SOAP response and handle the result.

```
WebServiceTemplate template = new WebServiceTemplate();
template.setDefaultUri("https://example.com/soap/CustomerService");
CustomerRequest request = new CustomerRequest("1001");
CustomerResponse response = (CustomerResponse) template.marshalSendAndReceive(request);
// Process response
String customerName = response.getCustomer().getName();
```

**Technologies Used**
- Spring Web Services
- WebServiceTemplate
- JAXB (Marshalling)
- WSDL
- XML
- WS-Security

## 6. BENEFITS

**Interoperability**
Works across different platforms and technologies.

**Reliability**
Supports standards for security, transactions, and fault handling.

**Security**
Enterprise-grade security with WS-Security and SSL/TLS.

**Reusability**
Existing enterprise services can be consumed easily.

**Loose Coupling**
Systems remain independent and loosely coupled.

## 7. BEST PRACTICES

- Always use HTTPS/TLS for secure communication.
- Use WS-Security for authentication and message protection.
- Implement timeouts and retry mechanisms.
- Log requests and responses (with sensitive data masked).
- Validate input and handle SOAP faults effectively.
- Monitor and track integration performance and errors.
- Keep WSDLs and dependencies version controlled.

# INTEGRATING SOAP SERVICES (2 OF 2)

*A sample Spring WS client, best practices, and how to handle SOAP faults from a downstream partner service.*

**Sample SOAP Client Code (Spring WS)**

```
WebServiceTemplate template = new WebServiceTemplate();
template.setDefaultUri("https://partner.example.com/soap/CustomerService");

QName qname = new QName("http://northstar.com/crm/customer", "getCustomerRequest");
GetCustomerRequest request = new GetCustomerRequest();
request.setCustomerId("CUS-1001");
GetCustomerResponse response = (GetCustomerResponse) template.marshalSendAndReceive(qname, request);
```

## BEST PRACTICES

- Always use HTTPS; validate XML with XSD before sending.
- Use WS-Security for authentication and integrity.
- Handle SOAP Faults and timeouts gracefully; cache WSDL locally to reduce latency.

## HANDLING FAULTS FROM A PARTNER

- Always catch SoapFaultClientException around marshalSendAndReceive calls.
- Log the fault code, message, and correlation id -- never assume the call succeeded.
- Watch for WSDL changes, version drift, and large-payload latency in production.

**KEY TAKEAWAY** A resilient SOAP client treats faults, timeouts, and WSDL drift as first-class failure modes, not edge cases.

# ENTERPRISE SOAP BEST PRACTICES (1 OF 2)

*Following best practices ensures SOAP web services are secure, reliable, maintainable, and well-integrated.*

| # | Practice | What It Means |
|---|---|---|
| **1** | Use Contract-First Approach | Define WSDL and XSD first to establish a clear, stable contract. |
| **2** | Follow Naming Conventions | Use consistent, meaningful names for operations, messages, and types. |
| **3** | Keep Contracts Simple & Versioned | Avoid over-complicated schemas; version your services (e.g. /v1, /v2). |
| **4** | Validate with XSD | Always validate XML requests and responses against the schema. |
| **5** | Use WS-Security Standards | Implement authentication, integrity, and encryption using WS-Security. |

**KEY TAKEAWAY** Contract discipline and schema validation are the foundation everything else -- security, reliability, observability -- builds on.

# ENTERPRISE SOAP BEST PRACTICES (2 OF 2)

*Apply these practices consistently to deliver reliable, maintainable, well-observed enterprise SOAP services.*

| # | Practice | What It Means |
|---|---|---|
| **6** | Enforce HTTPS | Always use SSL/TLS for transport security, in addition to WS-Security. |
| **7** | Handle Faults Gracefully | Use SOAP Faults to return meaningful, non-leaking error information. |
| **8** | Use Idempotent Operations | Design operations to be idempotent so retries are safe. |
| **9** | Centralize Logging & Correlation | Log requests, responses, and faults; include correlation IDs for tracing. |
| **10** | Monitor, Alert & Document | Track service health, latency, and errors; keep contracts documented. |

**KEY TAKEAWAY** Well-designed, secure, and observable SOAP services ensure long-term success in enterprise integration.

# TRY IT YOURSELF — EXERCISE 6: LAB 24 READINESS CHECKLIST

*Capstone check -- confirm the Lab 23 REST baseline is understood before you open the Spring-WS lab.*

## STEP / WHAT TO DO

| Step | What To Do |
|---|---|
| **Goal** | Verify the Lab 23 REST baseline is understood before starting the Spring-WS lab. |
| **File** | notes/lab24-readiness.md (in examples/module-24-exercises/). |
| **Dependency** | Note that Lab 23's Boot app + CustomerService is required before Lab 24 begins. |
| **Path** | Write the copy target: examples/lab24-crm. |
| **Evidence correlation** | Record lab24-001 for SOAP evidence (REST may still use lab-request-001). |
| **Deliverable** | Readiness note ties SOAP to the Boot baseline; list Kafka and React as Week 4, not Lab 24. |

## Readiness Checklist (notes/lab24-readiness.md)

```
[ ] Lab 23 Boot app + CustomerService exist and run          [ ] Copy target confirmed: examples/lab24-crm
[ ] Correlation ids noted: lab24-001 (SOAP), lab-request-001 (REST)    [ ] Kafka + React deferred to Week 4
```

**TRY IT NOW** Complete Exercise 6 before continuing -- see exercises/exercise-06-lab24-readiness.md.

# LAB OVERVIEW -- SOAP WEB SERVICE ENDPOINTS

*Lab 24: SOAP Web Service Endpoints — Northstar CRM Spring-WS*

## BUSINESS SCENARIO

- A regional billing partner only integrates via SOAP/XML and must create, get, update status, and list customers using the Lab 13 contract.
- Ship Spring-WS beside the Lab 23 REST API: live WSDL from XSD, four operations, CLIENT faults for not-found/duplicate, UsernameToken required.
- Protocol may differ; business rules must not -- both protocols delegate to the same CustomerService.

## LEARNING OBJECTIVES

- Explain contract-first SOAP and why the XSD -- not Java -- is the source of truth.
- Author an XSD and let Spring-WS generate WSDL dynamically from it.
- Implement @Endpoint with @PayloadRoot / @RequestPayload / @ResponsePayload.
- Translate BusinessException subtypes into structured SOAP faults.
- Configure a minimal WS-Security UsernameToken interceptor.

## WHAT YOU BUILD

- Copy lab23-crm to lab24-crm; add Spring-WS + jaxb2 plugin; author customer.xsd; configure MessageDispatcherServlet + DefaultWsdl11Definition; implement CustomerEndpoint with four @PayloadRoot operations; add SOAP fault mapping and a UsernameToken interceptor; verify with curl + MockWebServiceClient; evidence with correlation lab24-001.

**KEY TAKEAWAY** Fixtures CUS-1001 (Amina Khan, ACTIVE), CUS-1002 (Ravi Singh, PROSPECT), and CUS-9999 (not-found) anchor every example.

# LAB TASKS AND DELIVERABLES

*Northstar CRM Spring-WS -- implementation steps, deliverables, and the evaluation rubric.*

| # | Implementation Step |
|---|---|
| 1 | Branch Lab 23 and add Spring-WS + jaxb2-maven-plugin dependencies. |
| 2 | Author customer.xsd and generate JAXB request/response classes. |
| 3 | Configure MessageDispatcherServlet and a live /ws/customer.wsdl. |
| 4 | Map JAXB types and implement CustomerEndpoint's four operations. |
| 5 | Share exceptions and map SOAP faults (CLIENT for not-found/duplicate). |
| 6 | Add a Wss4jSecurityInterceptor for UsernameToken (WS-Security). |
| 7 | Prove REST and SOAP share one CustomerService (same data, both protocols). |
| 8 | Automate with MockWebServiceClient tests + a partner evidence pack. |

## EXPECTED DELIVERABLES

- CustomerEndpoint with four SOAP operations + customer.xsd / live WSDL.
- JAXB generation + CustomerSoapMapper; SOAP fault mapping.
- Working UsernameToken interceptor (lab secret only).
- requests/ sample XML + fault cases; CustomerEndpointTest green twice.
- Evidence that REST and SOAP share CustomerService; README runbook.

## RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security 10 · Docs 10

**KEY TAKEAWAY** Duplicating business rules or skipping WS-Security both lose core marks -- committing real secrets is an honor violation.

# MODULE 24 SUMMARY

*SOAP Web Service Endpoints and Request Mapping -- what we covered and how it fits together.*

## KEY CONCEPTS COVERED

### Contract-First & Recap

SOAP architecture, WSDL, and XML Schema as the source of truth.

### Endpoints & Mapping

@Endpoint and @PayloadRoot route SOAP requests to Java methods.

### Lifecycle & Marshalling

Request/response lifecycle and XML marshalling/unmarshalling.

### SOAP Faults

Structured faultcode/faultstring errors instead of stack traces.

### WS-Security

UsernameToken and message-level security concepts.

### Enterprise Best Practices

Contracts, security, reliability, and observability for SOAP.

## MODULE FLOW RECAP

1. **Contract (WSDL/XSD)**
2. **@Endpoint/@PayloadRoot**
3. **Marshalling**
4. **Faults**
5. **WS-Security**

**KEY TAKEAWAY** You can now design, build, secure, and integrate production-ready SOAP endpoints with Spring WS -- contract-first, end to end.

# KNOWLEDGE CHECK (1 OF 3)

*Test your understanding of SOAP fundamentals, WSDL, and Spring WS endpoint annotations.*

**1. What is the role of WSDL in a SOAP web service?**

**A.** To describe the service implementation

**B. To define the service contract, operations, and messages**

**C.** To handle service security

**D.** To store service logs

**2. Which annotation is used to define a SOAP endpoint in Spring WS?**

**A.** @Service

**B. @Endpoint**

**C.** @WebService

**D.** @Component

**3. Which annotation maps an incoming request to a specific XML namespace and local part?**

**A.** @RequestMapping

**B. @PayloadRoot**

**C.** @Namespace

**D.** @XmlRootElement

**4. What is the purpose of XML Marshalling in Spring WS?**

**A.** Convert XML to Java objects

**B. Convert Java objects to XML**

**C.** Validate XML against XSD

**D.** Encrypt XML messages

**KEY TAKEAWAY** WSDL, @Endpoint, and @PayloadRoot are the foundation of every Spring WS contract-first service.

# KNOWLEDGE CHECK (2 OF 3)

*Four more questions on unmarshalling, WS-Security, contract-first design, and integration.*

**5. Which of the following is TRUE about XML Unmarshalling?**

**A.** It converts Java objects to XML

**B. It converts XML into Java objects**

**C.** It is used only for request messages

**D.** It is not required in Spring WS

**6. WS-Security primarily provides which capabilities?**

**A.** Compression of SOAP messages

**B. Authentication, integrity, and confidentiality**

**C.** Message routing and queuing

**D.** Database encryption

**7. Which approach is recommended for enterprise SOAP service development?**

**A.** Code-First approach

**B. Contract-First approach**

**C.** Database-First approach

**D.** Manual-First approach

**8. What is a key benefit of integrating SOAP services with WebServiceTemplate?**

**A.** Faster UI rendering

**B. Loose coupling and reuse across systems**

**C.** Elimination of databases

**D.** Replacing REST APIs

**KEY TAKEAWAY** Contract-first design and WS-Security are what make a SOAP integration enterprise-ready.

# KNOWLEDGE CHECK (3 OF 3)

*Four scenario questions on SOAP faults, UsernameToken security, and shared business rules.*

**9. A CustomerNotFoundException is thrown inside CustomerEndpoint. What should the client receive?**

**A.** An HTTP 500 with a full Java stack trace

**B. A SOAP Fault with faultcode Client and a clear faultstring**

**C.** A silent empty response

**D.** The connection is simply closed

**10. True or False: A request without a valid UsernameToken should be rejected before business logic runs.**

**A. True — enforced before the endpoint method runs**

**B.** False

**11. Why must CustomerEndpoint delegate to CustomerService instead of re-implementing business rules?**

**A.** SOAP endpoints cannot call service classes

**B. So REST and SOAP never diverge on business rules**

**C.** It makes the WSDL generate faster

**D.** JAXB requires it

**12. True or False: HTTPS alone is a complete substitute for WS-Security UsernameToken.**

**A.** True

**B. False — transport security and message-level security solve different problems**

**KEY TAKEAWAY** SOAP faults, WS-Security, and a single shared service are the three pillars of a trustworthy enterprise SOAP endpoint.

*"Contract-first, fault-aware, and secured --*
*SOAP done the enterprise way."*

MODULE 24 COMPLETE

# SOAP Web Service Endpoints and Request Mapping

You can now design, build, secure, and integrate contract-first SOAP endpoints with Spring WS -- @Endpoint, @PayloadRoot, marshalling, faults, and WS-Security.